



INTERVERSE - AN INTERVIEW PLATFORM

PROJECT REPORT

Submitted by

Aditya Panja

Autonomy Roll No : 12624010003

Soumi Chandra

Autonomy Roll No : 12624010049

Subhadip Saha

Autonomy Roll No : 12624010055

in partial fulfillment for 4th Semester Major Project

of

MASTER OF COMPUTER APPLICATION

DEPARTMENT OF COMPUTER APPLICATIONS

HERITAGE INSTITUTE OF TECHNOLOGY, KOLKATA

MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY,

WEST BENGAL

May 2026



**DEPARTMENT OF COMPUTER APPLICATIONS
HERITAGE INSTITUTE OF TECHNOLOGY, KOLKATA**

BONAFIDE CERTIFICATE

Certified that this project report titled “Interverse - An Interview Platform” is the bonafide work carried out by Aditya Panja, Soumi Chandra and Subhadip Saha students of MCA 4th Semester, Department of Computer Applications, Heritage Institute of Technology, Kolkata, under supervision of Prof. (Dr.) Surajit Giri.

Prof. (Dr.) Souvik Basu
Head of Department,
Department of Computer Applications
Heritage Institute of Technology, Kolkata

Dr. Surajit Giri
Coordinator
Department of Data Science and AI
K. Mission Residential College (Autonomous)
Narendrapur, Kolkata - 700103

Signature: S. Giri
Date: 20/05/2020

Prof. (Dr.) Surajit Giri
Co-ordinator of Data Science and AI department
Ramakrishna Mission Residential College (Autonomous), Narendrapur

Examiner

DECLARATION BY STUDENT

This declaration confirms that the report titled “Interverse - An Interview Platform” has been created by us. We certify that the content of this report is original and free from plagiarism. All sources of information used in this project have been properly cited. We accept full responsibility for any instances of plagiarism that may be discovered in this report.

Name : Aditya Panja

Autonomy Roll No : 12624010003

Name : Soumi Chandra

Autonomy Roll No : 12624010049

Name : Subhadip Saha

Autonomy Roll No : 12624010055

ACKNOWLEDGEMENT

We express our deepest gratitude to Prof. (Dr.) Surajit Giri , our project supervisor, for his invaluable guidance and steadfast supervision, which were crucial to the successful completion of our project. We also wish to extend our sincere thanks to the Department of Computer Applications for granting us easy access to all necessary facilities, and to all the professors for their guidance and motivation.

Name : Aditya Panja

Autonomy Roll No : 12624010003

Name : Soumi Chandra

Autonomy Roll No : 12624010049

Name : Subhadip Saha

Autonomy Roll No : 12624010055

TABLE OF CONTENTS

Serial Number	Topics	Page Number
1	Abstract	6
2	Introduction	7
3	Objective	8
4	Purpose	9
5	Past Works	10
6	Models	11 - 15
7	Datasets	16 – 18
8	Methodology	19 – 22
9	Technologies Used	23 - 24
10	System Architecture	25 - 37
11	Results and Discussion	38-41
12	Future Scope	42 - 43
13	Conclusion	44 - 45
14	References	46

a. Abstract

INTERVERSE is a web based interview platform developed to provide users with realistic and flexible interview practice experience. The system conducts domain specific interviews, analyzes answers based on meaning and context, and changes question difficulty based on how well the user performs. Unlike traditional interview platforms that mostly rely on keyword checking, INTERVERSE understands the actual meaning of answers and evaluates them in a more natural and human like way.

The platform is built using React.js for frontend development and FastAPI for backend development and SQLite is used for database management. Transformer based Natural Language Processing models such as Sentence BERT (SBERT) and RoBERTa are added to the platform to improve answer evaluation. These models help the platform understand the actual meaning, relevance, and correctness of the response.

INTERVERSE also includes other features such as JWT based authentication, interview history tracking, text to speech functionality, difficulty adjustment, and real-time chat interaction to create a more realistic interview environment. The system is designed to help users improve their technical skills, communication skills and confidence through smart feedback and regular practice.

The project shows how modern NLP techniques and full stack web technologies can be used practically in developing a useful interview practice and performance tracking platform.

b.Introduction

Technical interviews are one of the most important stages of recruitment process in technology related fields. Many students and job seekers face difficulties while preparing for interviews because they often do not get enough real interview practice and useful feedback. Existing interview preparation platforms mostly focus on coding problems or predefined question answer patterns, which do not properly recreate the experience of an actual technical interview.

Most traditional interview platforms check answers mainly through keyword matching. Although these approaches are easy to build, they often cannot understand the actual meaning of an answer. As a result, candidates may receive incorrect evaluations even when their responses are technically correct but written differently from the expected answer. This creates a need for more intelligent systems capable of understanding semantic meaning and logical correctness.

INTERVERSE is developed to solve these limitations by providing an interactive web based interview platform that conducts domain specific technical interviews with contextual answer evaluation. The platform lets users attend interviews where the question difficulty changes automatically based on their performance. It also provides detailed feedback, interview history, and performance tracking to help users improve their preparation and communication skills.

The system combines several important technologies and concepts, including:

1. Natural Language Processing (NLP)
2. Semantic Similarity Analysis
3. Natural Language Inference
4. Adaptive Questioning
5. Authentication Systems
6. Real-Time Interaction
7. Performance Analytics

To evaluate answers more accurately, INTERVERSE uses transformer based NLP models such as Sentence BERT (SBERT) and RoBERTa. These models help the system understand the meaning of user answers instead of only checking keywords. The platform is developed using React.js for the frontend, FastAPI for backend services, and SQLite for database management.

The main objective of INTERVERSE is to create an useful interview practice environment that improves interview readiness, technical skills, communication skills, confidence, and overall performance through intelligent evaluation and adaptive interaction.

c. Objective

The main goal of INTERVERSE is to develop an interactive interview platform that helps users to improve their preparation through real interview like practice and contextual answer evaluation. The system provides an organized platform where users can practice domain based interviews and receive useful feedback based on their answers.

Another important objective is to overcome the limitations of traditional keyword based checking systems by using transformer-based NLP models for semantic and logical analysis. This helps the platform understand what the user actually means instead of only checking exact keywords.

The project also provides interviews where question difficulty changes automatically based on the user's performance. This helps to create a more personalized and interactive interview experience. In addition, the platform focuses on improving technical communication skills, confidence, and problem solving skills by providing performance tracking, interview history, and feedback after each session.

The major objectives of INTERVERSE include:

1. Conducting domain specific interviews
2. Evaluating answers using semantic similarity and logical checking
3. Implementing automatic difficulty adjustment
4. Providing real time interactive interview experience
5. Maintaining secure authentication and session handling
6. Generating interview reports and performance analysis
7. Improving user confidence and interview preparation through continuous practice

d.Purpose

The main purpose of INTERVERSE is to provide an interactive and realistic platform for technical interview preparation. The system is designed to help students, job seekers, and aspiring software professionals improve their interview skills through adaptive interview sessions and contextual answer evaluation.

Traditional interview preparation platforms mainly focus on coding practice or static question-answer methods, which often do not reflect real interview situations. INTERVERSE aims to overcome these limitations by creating a more dynamic and conversational interview environment where users can practice domain-specific technical interviews and receive meaningful feedback based on their responses.

Another important purpose of the platform is to evaluate answers semantically instead of relying only on exact keyword matching. By using transformer-based NLP models such as SBERT and RoBERTa, the system can understand contextual meaning, logical correctness, and conceptual similarity in user responses. This makes the evaluation process more accurate and realistic.

INTERVERSE also aims to:

- Improve technical communication skills
- Increase interview confidence
- Provide adaptive difficulty progression
- Track interview performance and analytics
- Create a realistic interview simulation experience
- Help users identify strengths and weaknesses

The platform is developed to combine modern web technologies and Natural Language Processing techniques into a practical system that supports effective interview preparation and continuous learning.

e. Past Work

Several online platforms have been developed to assist students and job seekers in preparing for technical interviews. Well-known platforms such as [LeetCode](#), [HackerRank](#), [InterviewBit](#), and [Pramp](#) mainly concentrate on coding exercises, aptitude assessments, and predefined interview questions. These platforms are widely used for improving programming knowledge and problem-solving abilities required in technical recruitment processes.

Most existing interview preparation systems primarily rely on:

- Coding practice problems
- Fixed assessment patterns
- Static question repositories
- Keyword-based or manually evaluated answers

Although these systems are effective for practicing technical concepts and coding challenges, they often fail to simulate a realistic interview environment. In many cases, answer evaluation depends heavily on exact keyword matching, which limits the system's ability to understand conceptually correct responses written in different ways. Because of this limitation, candidates may receive inaccurate evaluations even when their answers are technically valid.

Another drawback of traditional interview platforms is the lack of adaptive questioning mechanisms. The difficulty level of questions generally remains constant and does not adjust according to the candidate's performance during the interview session. Furthermore, many platforms provide limited analytical feedback and lack interactive communication features that are important in real interview situations.

Recent advancements in Natural Language Processing (NLP) and transformer-based deep learning architectures have significantly improved contextual understanding and semantic analysis in language processing applications. Modern transformer models such as:

- BERT
- Sentence-BERT (SBERT)
- RoBERTa

have introduced advanced methods for semantic similarity measurement, contextual embedding generation, and logical language understanding. These models can analyze the meaning and intent behind textual responses instead of depending solely on exact word matching techniques.

INTERVERSE is developed by utilizing these modern NLP technologies to create a more intelligent and interactive interview preparation platform. The system integrates semantic answer evaluation, adaptive difficulty progression, and real-time interaction features to provide a more realistic and effective technical interview practice experience compared to conventional interview preparation systems.

f. Models

INTERVERSE uses transformer-based Natural Language Processing (NLP) models to evaluate interview answers intelligently and contextually. Instead of relying only on keyword matching, the system analyzes semantic meaning and logical correctness of user responses. Two major models are integrated into the platform: Sentence-BERT (SBERT) using the all-MiniLM-L6-v2 model and RoBERTa using the roberta-large-mnli model.

Sentence-BERT (SBERT)

Sentence BERT (SBERT) is used in INTERVERSE for semantic similarity evaluation. Traditional keyword-based systems cannot properly determine whether two answers convey the same meaning when different words or sentence structures are used. SBERT solves this problem by converting sentences into vector embeddings and comparing their semantic meaning.

SBERT is built on the BERT architecture and follows a Siamese Network approach to generate meaningful sentence representations. In INTERVERSE, it helps evaluate whether the user's answer is conceptually similar to the expected answer stored in the dataset.

Purpose in INTERVERSE

- Evaluate conceptual correctness of answers
- Understand semantic meaning instead of exact keywords
- Support flexible and natural language responses
- Improve accuracy of interview evaluation

Model Used: all-MiniLM-L6-v2

The all-MiniLM-L6-v2 model is a lightweight and efficient SBERT model optimized for semantic text similarity tasks. It is trained on large-scale sentence pair datasets and designed to generate high-quality sentence embeddings with low computational cost.

Important Features

- Generates 384-dimensional sentence embeddings
- Fast inference speed with low memory usage
- Optimized for semantic similarity analysis
- Performs effectively on paraphrased responses
- Suitable for real-time applications

Working Process

1. The expected answer is converted into an embedding vector.
2. The user's answer is converted into another embedding vector.
3. Cosine similarity is calculated between the two vectors.
4. A similarity score is generated based on semantic closeness.

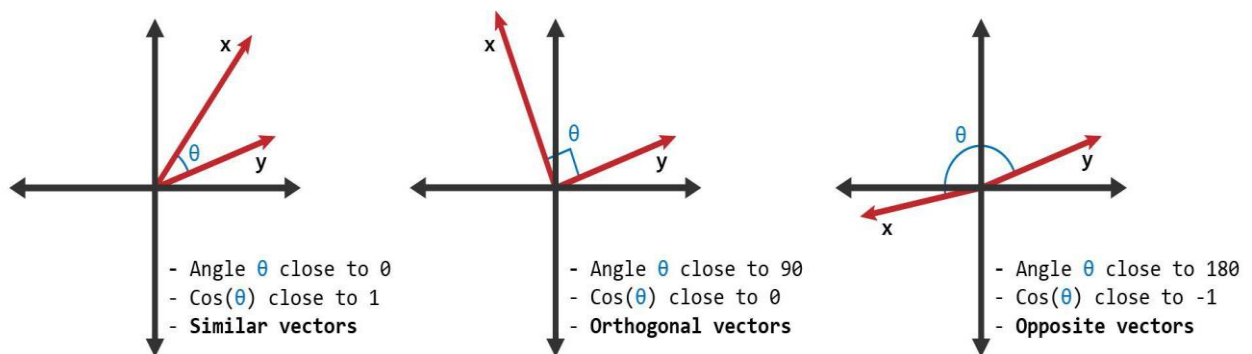
Cosine Similarity Formula

$$\text{Cosine Similarity} = \frac{A \cdot B}{\|A\| \|B\|}$$

Where:

A = User answer embedding

B = Expected answer embedding

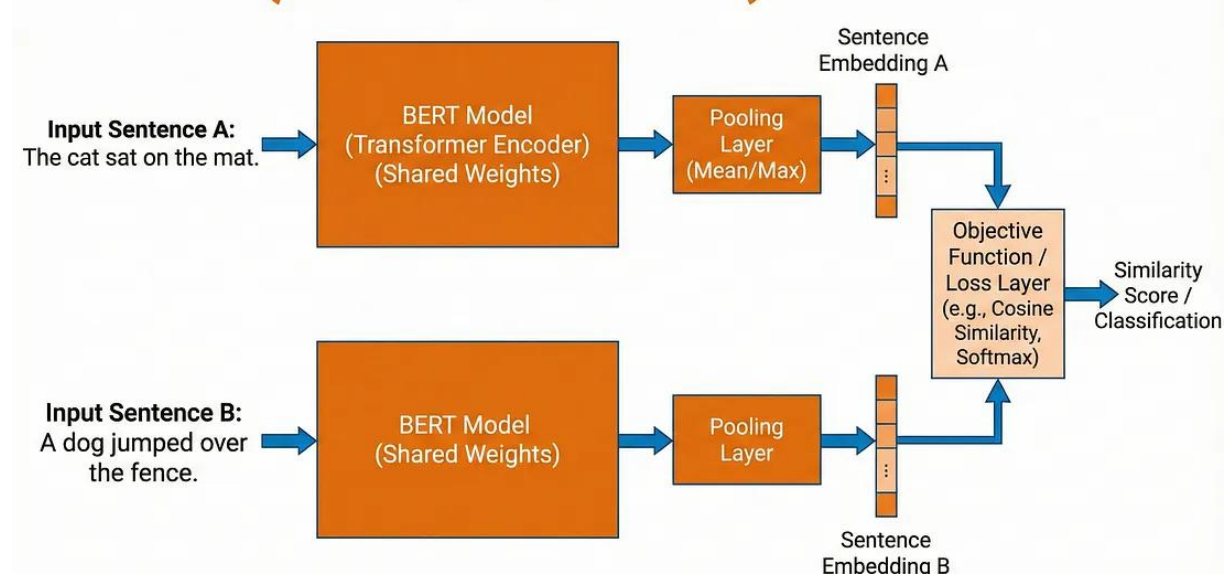


The similarity score ranges from:

0 → Completely unrelated

1 → Highly similar

SBERT (Sentence-BERT) Architecture



Advantages

1. Understands contextual meaning effectively
2. Handles descriptive and technical answers well
3. Recognizes paraphrased responses
4. More accurate than traditional keyword matching

RoBERTa (Robustly Optimized BERT Approach)

RoBERTa is used in INTERVERSE for Natural Language Inference (NLI) based answer validation. While SBERT measures semantic similarity, RoBERTa verifies whether the user's answer logically supports the expected answer.

RoBERTa is an optimized version of BERT developed for stronger contextual understanding and inference tasks. It is trained on large-scale textual datasets and fine-tuned for classification tasks such as entailment, contradiction, and neutrality.

Purpose in INTERVERSE

- Perform logical verification of answers
- Detect whether responses support the expected concept
- Reduce incorrect scoring caused by superficial similarity
- Improve reliability of evaluation results

Model Used: roberta-large-mnli

The roberta-large-mnli model is a transformer-based Natural Language Inference model fine-tuned on the Multi-Genre Natural Language Inference (MNLI) dataset. It is designed to identify logical relationships between two sentences.

Important Features

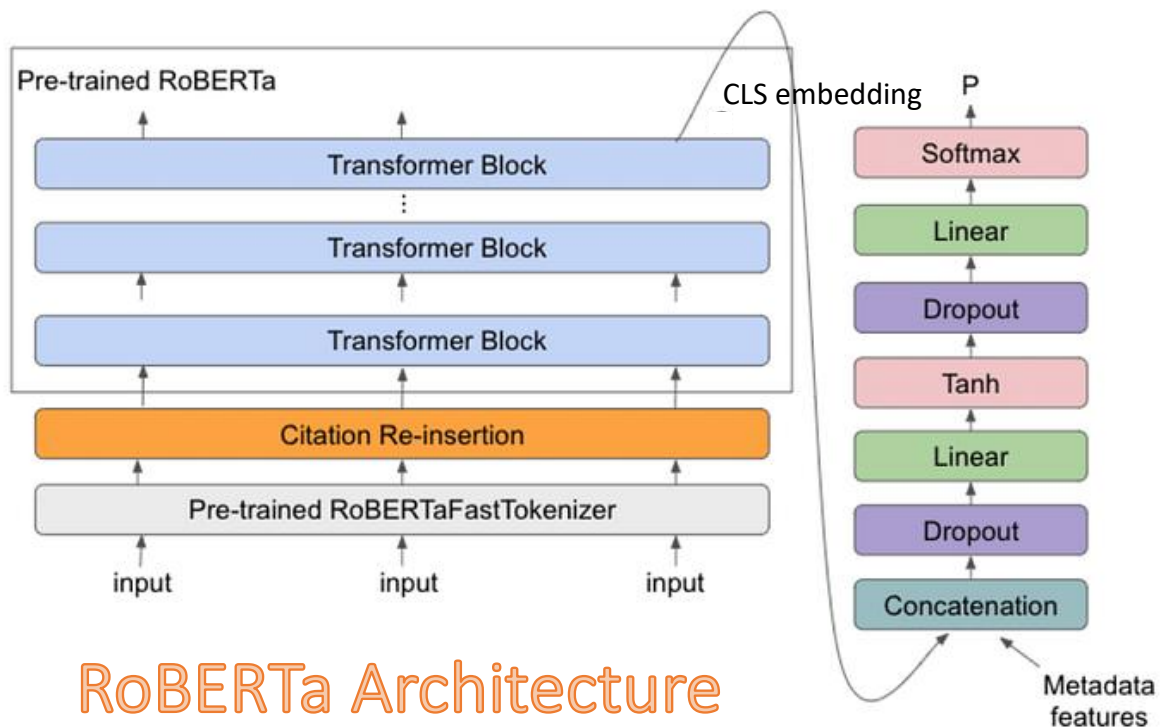
- Based on the large RoBERTa transformer architecture
- Fine-tuned for entailment classification tasks
- Predicts:
 - 1) Entailment
 - 2) Neutral
 - 3) Contradiction
- Strong contextual reasoning capability
- Handles complex technical language effectively
- Architecture Details :
 - Model Type : RoBERTa-large
 - Transformer Layers : 24
 - Hidden Size : 1024

Attention Heads :16

Parameters : ~355 Million

Working Process

1. The expected answer is treated as the premise.
2. The user's answer is treated as the hypothesis.
3. RoBERTa analyzes the relationship between them.
4. The model predicts whether the answer:
 - i. Supports the concept (Entailment)
 - ii. Is partially related (Neutral)
 - iii. Contradicts the expected answer (Contradiction)



RoBERTa Architecture

Advantages

1. Strong contextual understanding
2. Better logical reasoning capability
3. Handles complex sentence structures
4. Improves evaluation accuracy and reliability

Combined Evaluation Strategy

INTERVERSE combines SBERT semantic similarity analysis with RoBERTa logical inference validation to produce accurate and reliable interview evaluations.

Evaluation Pipeline

1. User submits an answer
2. SBERT calculates semantic similarity score
3. RoBERTa validates logical correctness
4. Scores from both models are combined
5. Final feedback and adaptive difficulty updates are generated

This hybrid evaluation strategy improves contextual understanding, scoring fairness, and overall reliability of the interview assessment process.

g.Datasets

INTERVERSE uses a structured interview dataset stored in CSV format for managing technical interview questions and expected answers. The dataset acts as the primary knowledge source for generating domain-specific interview sessions and evaluating user responses. It is designed to support adaptive questioning, semantic evaluation, and difficulty-based interview progression.

The dataset contains a total of approximately 1260 interview questions distributed across 21 technical and non-technical domains. Each record in the dataset includes the following fields:

- Question Number
- Question
- Answer(s)
- Category
- Difficulty Level

The expected answer field contains multiple acceptable answers stored in list format. This allows the system to evaluate responses more flexibly and improves semantic matching accuracy during answer evaluation.

Difficulty Levels

Questions are categorized into three difficulty levels:

- Easy
- Medium
- Hard

This structure supports the adaptive difficulty engine of INTERVERSE, where question complexity changes according to user performance during the interview session.

	A	B	C	D	E
1	Question Number	Question	Answer	Category	Difficulty
			["A data structure is a way of organizing and storing data so that it can be accessed and modified efficiently.", "Data structures are specialized formats used to store, manage, and arrange data in a computer for efficient processing.", "A data structure refers to the logical or mathematical model used to organize data elements and their relationships.", "It is a method of structuring data in memory to perform operations like insertion, deletion, and searching effectively.", "Data structures are techniques used in computer science to organize data in a way that optimizes performance and resource usage."]		
2		1 What is a Data Structure?		Data Structures & Algorithms (DSA)	Easy
3		2 What is an Algorithm?		Data Structures & Algorithms (DSA)	Easy
4		3 What is the difference between an array and a linked list?		Data Structures & Algorithms (DSA)	Easy
5		4 What is Time Complexity?		Data Structures & Algorithms (DSA)	Easy
6		5 What is Space Complexity?		Data Structures & Algorithms (DSA)	Easy
7		6 What is Big-O Notation?		Data Structures & Algorithms (DSA)	Easy
8		7 What is a Stack?		Data Structures & Algorithms (DSA)	Easy
9		8 What is a Queue?		Data Structures & Algorithms (DSA)	Easy
10		9 What is a Linked List?		Data Structures & Algorithms (DSA)	Easy
11		10 What is a Binary Tree?		Data Structures & Algorithms (DSA)	Easy

Domains Included

The dataset covers multiple interview domains such as:

- 1) Artificial Intelligence / Machine Learning
- 2) Backend Development
- 3) Cloud Computing
- 4) Computer Network
- 5) Computer Organization & Architecture
- 6) Cybersecurity
- 7) Data Structures & Algorithms (DSA)
- 8) DBMS
- 9) Frontend Development
- 10) Full Stack Web Development
- 11) HR & Behavioral
- 12) NLP
- 13) OOPS
- 14) Operating System
- 15) Programming Languages (C)
- 16) Programming Languages (C++)
- 17) Programming Languages (Java)
- 18) Programming Languages (Python)
- 19) Software Engineering
- 20) System Design
- 21) Testing & Debugging

This wide domain coverage allows the platform to conduct diverse and specialized interview sessions based on user preferences.

Dataset Processing and Preprocessing

The dataset is processed using Python libraries such as Pandas and AST parsing utilities. Several preprocessing and cleaning techniques are applied before using the dataset in the interview system.

The preprocessing steps include:

- Reading and organizing CSV data using Pandas
- Cleaning and standardizing column names
- Removing unnecessary or incomplete entries
- Converting answer lists from string format using AST parsing
- Normalizing category and difficulty labels
- Filtering duplicate or repeated questions
- Handling encoding issues during dataset loading

These preprocessing techniques improve data consistency, reduce errors during evaluation, and ensure smooth backend processing.

Dynamic Question Selection

Questions are dynamically selected during interview sessions based on:

- User-selected domain
- Current difficulty level
- Previously asked questions

This filtering mechanism prevents repetition and ensures that users receive relevant and diverse interview questions throughout the session.

The dataset design and preprocessing pipeline play an important role in improving the accuracy, adaptability, and overall effectiveness of the INTERVERSE interview platform.

h.Methodology

INTERVERSE is designed using a structured client-server architecture that integrates frontend interaction, backend processing, database management, authentication, and transformer-based Natural Language Processing (NLP) techniques. The platform aims to simulate a real-world technical interview environment where users can practice domain-specific interviews, receive intelligent answer evaluation, and monitor their performance through analytics. The entire system is divided into multiple interconnected modules to ensure scalability, maintainability, and smooth user interaction.

1. Frontend Development

The frontend of INTERVERSE is developed using React.js to create a modern and responsive user interface. The frontend communicates with backend APIs through HTTP requests and manages all user-side interactions during interview sessions.

The interface provides multiple modules such as:

- User login and registration pages
- Dashboard and interview management pages
- AI-based chat interview interface
- Interview history section
- Analytics and performance tracking pages
- Secure protected routes for authenticated users

Several interactive features are integrated into the frontend, including:

- Dynamic interview domain selection
- Real-time question and answer interaction
- Timer-controlled interview sessions
- Voice-based answer submission
- Speech synthesis for automated question narration
- Responsive UI for desktop and mobile devices

The frontend design focuses on usability, smooth navigation, and creating an interview experience similar to professional online interview platforms.

2. Backend Development

The backend of INTERVERSE is implemented using FastAPI, which supports asynchronous request handling and high-performance API development. The backend controls the main business logic of the platform, including authentication, session handling, question retrieval, answer evaluation, and analytics generation.

The backend system performs the following operations:

- User authentication and token verification
- Interview session creation and management
- Dynamic question filtering
- Semantic answer evaluation
- Adaptive difficulty adjustment
- Storage and retrieval of interview history
- Analytics generation for user performance

The backend follows a modular architecture divided into different layers such as:

- API routes
- Schemas and validation models
- Database models
- Service modules
- AI evaluation modules

This modular design improves code organization, scalability, debugging, and future expansion of the platform.

3. Database Design

SQLite is used as the primary database system for storing user and interview-related information. SQLAlchemy ORM is integrated with the backend to simplify database interactions and object-relational mapping.

The database mainly contains the following tables:

- Users table
- Interview sessions table
- Responses table

The database stores important information such as:

- User credentials and profile details
- Interview session information
- User responses and scores
- Difficulty progression records
- Historical interview data

This structured database design helps maintain consistency and supports efficient retrieval of user analytics and interview records.

4. Authentication System

INTERVERSE implements JWT (JSON Web Token) based authentication to secure user sessions and restrict unauthorized access to protected resources. User passwords are encrypted using the PBKDF2-SHA256 hashing algorithm before being stored in the database.

The authentication workflow consists of the following steps:

1. User registration or login
2. JWT token generation after successful authentication
3. Token verification for protected API routes
4. Authorized access to interview features and analytics

This mechanism improves platform security and ensures that only authenticated users can access personal interview data.

5. Question Management System

The interview question dataset is stored in CSV format and categorized according to:

- Question ID
- Question text
- Expected answers
- Domain category
- Difficulty level

During interview sessions, the backend dynamically filters questions based on:

- Selected interview domain
- Current difficulty level
- Previously asked questions

This process prevents duplicate questions and ensures a varied interview experience throughout the session.

6. Semantic Evaluation System

INTERVERSE evaluates answers using transformer-based NLP models instead of traditional keyword-matching methods. The system analyzes the contextual meaning of the candidate's response and compares it with multiple expected answers.

The semantic evaluation process includes:

- Sentence embedding generation using SBERT
- Cosine similarity calculation between answers
- Natural Language Inference (NLI) using RoBERTa
- Contextual correctness verification

- Final score and feedback generation

This approach enables the platform to recognize conceptually correct answers even if the wording differs from the expected response.

7. Adaptive Difficulty Engine

The platform includes an adaptive difficulty mechanism that dynamically changes question complexity based on user performance during the interview session.

The interview difficulty levels include:

- Easy
- Medium
- Hard

If a user performs consistently well, the system increases the difficulty level and presents more advanced questions. Similarly, if performance decreases, easier questions are selected. This adaptive strategy creates a more personalized and realistic interview environment.

8. Speech Interaction System

INTERVERSE integrates browser-based Speech API technologies to enhance interview interaction and improve communication practice.

The speech module provides features such as:

- Automatic voice narration of interview questions
- Voice-based answer input
- Real-time speech interaction
- Improved communication and listening practice

This functionality helps users simulate real interview conversations and makes the platform more interactive and engaging.

i. Technologies Used

INTERVRSE is developed using modern full-stack web technologies, database systems, authentication mechanisms, and transformer-based Natural Language Processing libraries. These technologies work together to provide a scalable, interactive, and intelligent interview simulation platform.

1. React.js

React.js is used for frontend development of INTERVRSE. It provides a dynamic and responsive user interface for authentication pages, dashboard modules, interview chat interfaces, analytics sections, and profile management. Its component-based architecture improves code reusability, maintainability, and user interaction.

2. FastAPI

FastAPI is used for backend API development and server-side processing. It handles authentication, interview session management, question retrieval, answer evaluation, analytics generation, and communication between frontend and backend modules efficiently.

3. SQLite

SQLite is used as the database management system for storing user information, interview sessions, responses, scores, analytics data, and contact messages. It provides a lightweight and efficient storage solution suitable for the project requirements.

4. SQLAlchemy

SQLAlchemy is used as the Object Relational Mapper (ORM) for database operations. It simplifies database interaction by mapping Python classes to database tables and improves backend maintainability and scalability.

5. JWT (JSON Web Token)

JWT authentication is implemented to secure user sessions and protected routes. It verifies user identity and restricts unauthorized access to interview features, dashboards, and analytics pages.

6. Sentence Transformers

The Sentence Transformers library is used to integrate the SBERT model for semantic similarity evaluation. It helps compare user answers with expected answers contextually rather than relying only on keyword matching.

7. Hugging Face Transformers

The Hugging Face Transformers library is used to integrate transformer-based NLP models such as RoBERTa for Natural Language Inference (NLI) and contextual answer validation.

8. Pandas

Pandas is used for processing and managing CSV-based interview datasets. It assists in dataset cleaning, filtering, preprocessing, and question organization.

9. Recharts

Recharts is used for generating graphical analytics and performance visualizations in the dashboard. It helps display interview statistics, score trends, and domain-wise performance analysis.

10. CSS

CSS is used for designing responsive layouts, animations, themes, and interactive user interface components throughout the platform.

11. Web Speech API

The Web Speech API is used for browser-based speech recognition functionality. It converts spoken answers into text during interview sessions and enables voice-based interaction.

12. Speech Synthesis API

The Speech Synthesis API is integrated for automatic voice narration of interview questions. It improves interview realism and enhances user engagement.

13. Vite

Vite is used as the frontend build tool and development server. It provides faster startup time, efficient hot module replacement, and optimized frontend performance for React applications.

14. Scikit-learn

Scikit-learn is used for similarity computation and evaluation metric calculation. It is used for cosine similarity computation, accuracy measurement, precision, recall, F1-score evaluation, and confusion matrix generation.

15. PBKDF2-SHA256

PBKDF2-SHA256 is used for secure password hashing before storing user credentials in the database. It improves authentication security and protects user passwords from unauthorized access.

j. System Architecture

INTERVERSE follows a modular client-server architecture that combines frontend interaction, backend processing, database management, and NLP-based evaluation modules. The architecture is designed to support secure authentication, real-time interview interaction, adaptive questioning, and contextual answer evaluation. Each layer of the system works together to provide a smooth and efficient interview experience for users.

1. Frontend Layer

The frontend layer is developed using React.js and is responsible for handling all user interactions. It provides a responsive and interactive interface where users can participate in interview sessions and view their performance analytics.

Main frontend components include:

- Login and signup pages
- Dashboard interface
- Chat-based interview system
- Interview history and analytics section
- Protected routes for authenticated users

The frontend communicates with the backend through REST APIs and manages features such as:

- Real-time interview interaction
- Dynamic domain selection
- Timer-based interview sessions
- Voice-based answer input
- Speech synthesis for question narration
- Responsive and user-friendly interface design

2. Backend Layer

The backend layer is implemented using FastAPI, which provides high-performance API handling and modular architecture support. The backend manages the core functionality of the platform and acts as the communication bridge between the frontend, database, and NLP modules.

The backend handles:

- User authentication and authorization
- JWT token verification
- Interview session management
- Question filtering and generation

- Answer evaluation
- Adaptive difficulty updates
- Performance analytics generation

The backend is organized into modular components such as:

- Routes
- Schemas
- Services
- Database models

This structure improves scalability, maintainability, and readability of the application.

3. Database Layer

INTERVERSE uses SQLite as the database management system for storing user and interview-related information. SQLAlchemy ORM is used for database interaction and query management.

The database stores:

- User credentials
- Interview sessions
- User responses
- Evaluation scores
- Difficulty progression data
- Interview history and analytics

Main database tables include:

- Users
- Interview Sessions
- Responses

The database layer ensures proper data organization and supports long-term interview tracking and analytics generation.

4. NLP Evaluation Layer

The NLP evaluation layer is responsible for contextual answer analysis and intelligent response evaluation. INTERVERSE integrates transformer-based NLP models to evaluate answers semantically instead of relying only on keyword matching techniques.

The evaluation system includes:

- SBERT for semantic similarity analysis
- RoBERTa for logical inference validation

SBERT measures how closely the user's response matches the expected answer semantically, while RoBERTa verifies whether the response logically supports the expected concept. These

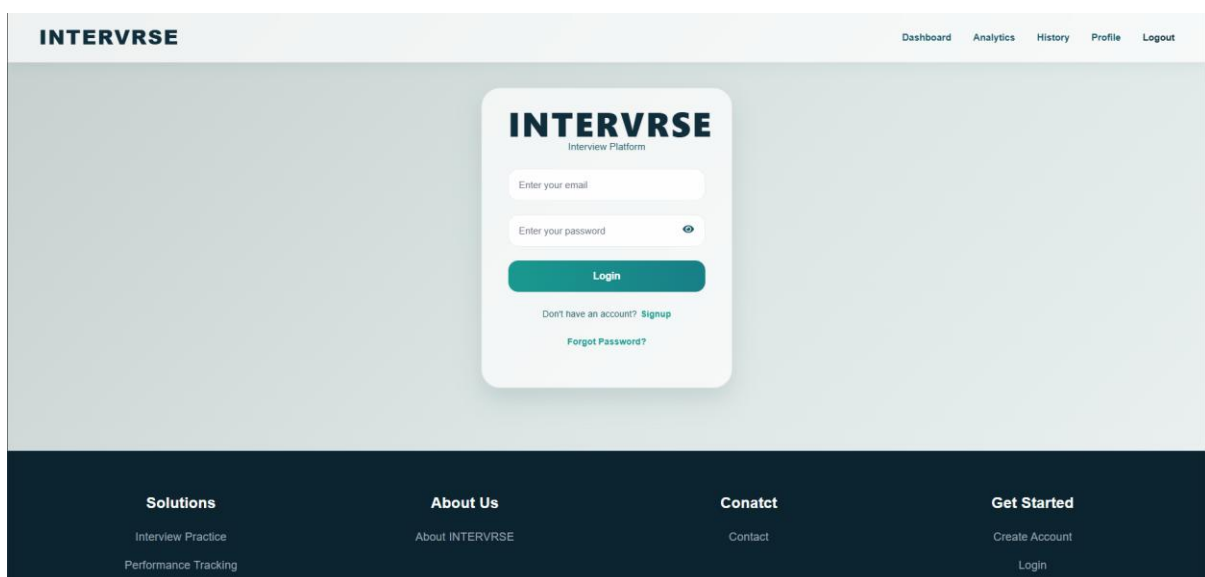
models work together to generate accurate evaluation scores and improve the reliability of interview assessment.

Workflow of INTERVERSE

The workflow of INTERVERSE follows a structured process that simulates a real technical interview environment.

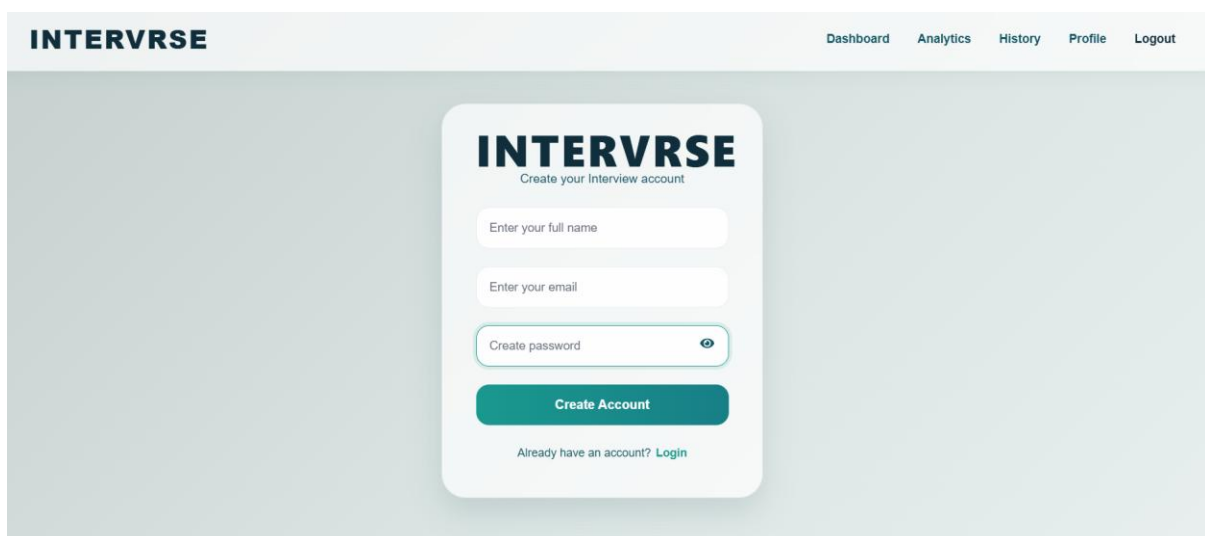
Step 1: User Authentication

Users first log into the platform using secure authentication. JWT token verification is used to protect interview routes and maintain secure user sessions.



The screenshot shows the User Login Page of the INTERVERSE platform. The page has a light blue header with the INTERVERSE logo on the left and navigation links (Dashboard, Analytics, History, Profile, Logout) on the right. The main content area is a light gray rectangle containing a white login card. The card has the INTERVERSE logo and 'Interview Platform' text at the top. Below this are two input fields: 'Enter your email' and 'Enter your password' (with a toggle icon). A teal 'Login' button is positioned below the password field. At the bottom of the card, there are two links: 'Don't have an account? Signup' and 'Forgot Password?'. The footer is a dark blue bar with four sections: 'Solutions' (Interview Practice, Performance Tracking), 'About Us' (About INTERVERSE), 'Contact' (Contact), and 'Get Started' (Create Account, Login).

User Login Page

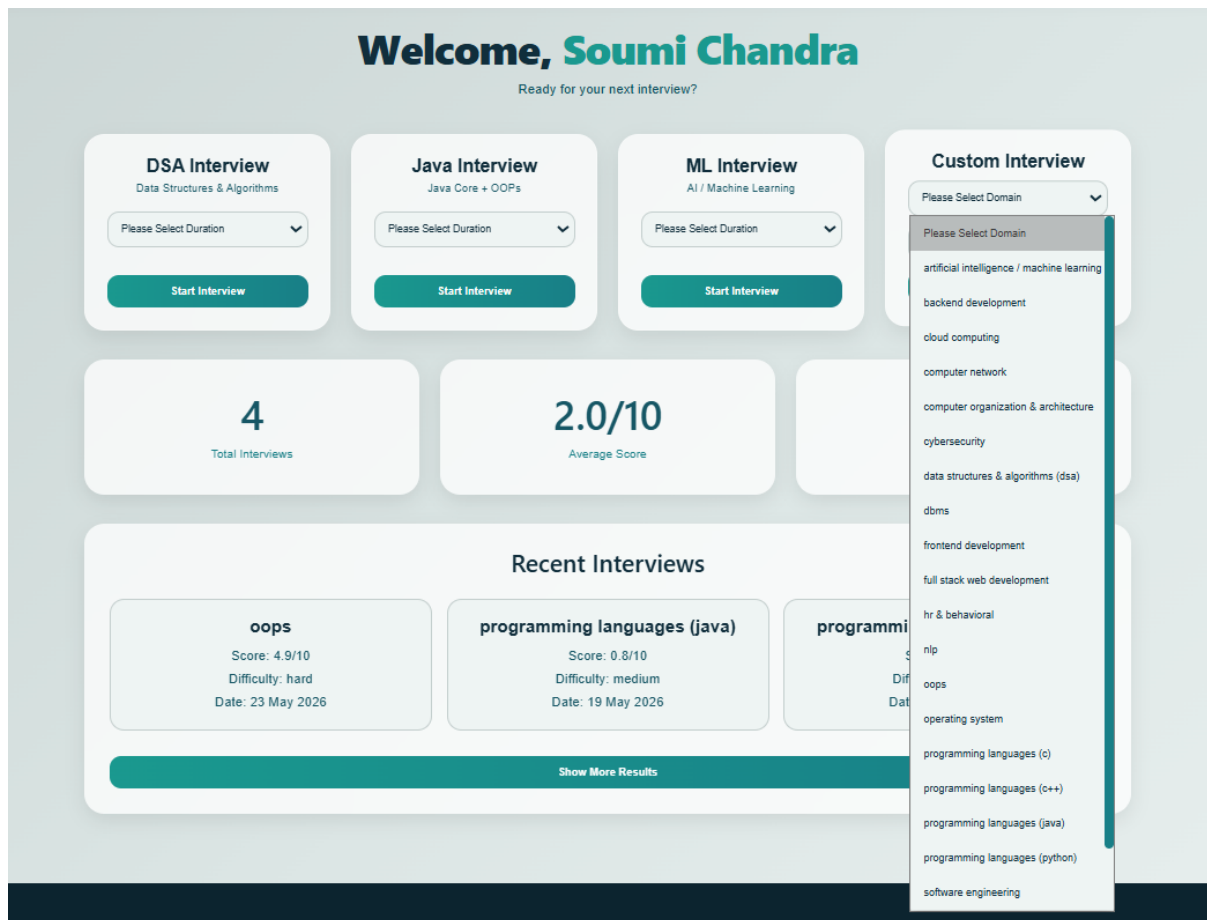


The screenshot shows the User Sign-up Page of the INTERVERSE platform. The page has a light blue header with the INTERVERSE logo on the left and navigation links (Dashboard, Analytics, History, Profile, Logout) on the right. The main content area is a light gray rectangle containing a white sign-up card. The card has the INTERVERSE logo and 'Create your Interview account' text at the top. Below this are three input fields: 'Enter your full name', 'Enter your email', and 'Create password' (with a toggle icon). A teal 'Create Account' button is positioned below the password field. At the bottom of the card, there is a link: 'Already have an account? Login'.

User Sign-up Page

Step 2: Interview Session Creation

After successful authentication, the user selects a preferred interview domain and duration. The backend then creates a new interview session for the user.



Dashboard

Step 3: Question Selection

The backend dynamically fetches domain-specific questions from the dataset according to:

- Selected domain
- Difficulty level (easy then medium and then hard)
- Previously asked questions

This prevents repetition and maintains interview diversity.

```

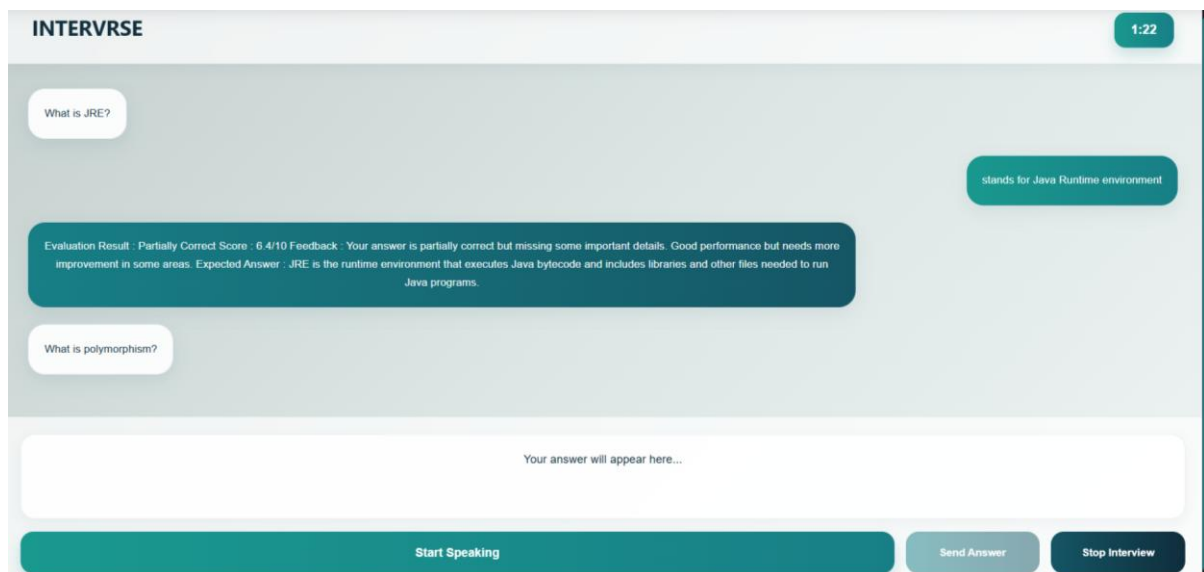
PS D:\Interverse\backend> python run.py
FETCHING QUESTION
DOMAIN: programming languages (java)
DIFFICULTY: easy
ASKED: 1
DOMAIN MATCH: 60
DIFFICULTY MATCH: 20
AFTER REMOVE ASKED: 19
QUESTION SELECTED
{'id': 61, 'question': 'What is Java?', 'answers': ['Java is a high-level, object-oriented programming language developed by Sun Microsystems that is widely used for building platform-independent applications.', 'It is a programming language and computing platform that allows developers to write code once and run it anywhere using the Java Virtual Machine (JVM).', 'Java is a versatile, class-based language known for its portability, security, and robustness, commonly used in web, mobile, and enterprise applications.', 'It is an object-oriented programming language that follows the principle of write once, run anywhere, making it platform-independent.', 'Java is a popular programming language used to develop applications ranging from desktop software to large-scale enterprise systems.', 'It is a strongly typed, compiled language that runs on the JVM and supports features like inheritance, polymorphism, and encapsulation.', 'Java is a general-purpose language designed to have minimal implementation dependencies and is widely used in Android and backend development.', 'It is a widely adopted programming language known for its cross-platform capability, reliability, and extensive library support.'], 'difficulty': 'easy'}
=====
QUESTION FOUND:
{'id': 61, 'question': 'What is Java?', 'answers': ['Java is a high-level, object-oriented programming language developed by Sun Microsystems that is widely used for building platform-independent applications.', 'It is a programming language and computing platform that allows developers to write code once and run it anywhere using the Java Virtual Machine (JVM).', 'Java is a versatile, class-based language known for its portability, security, and robustness, commonly used in web, mobile, and enterprise applications.', 'It is an object-oriented programming language that follows the principle of write once, run anywhere, making it platform-independent.', 'Java is a popular programming language used to develop applications ranging from desktop software to large-scale enterprise systems.', 'It is a strongly typed, compiled language that runs on the JVM and supports features like inheritance, polymorphism, and encapsulation.', 'Java is a general-purpose language designed to have minimal implementation dependencies and is widely used in Android and backend development.', 'It is a widely adopted programming language known for its cross-platform capability, reliability, and extensive library support.'], 'difficulty': 'easy'}
INFO: 127.0.0.1:64585 - "GET /interview/question/5 HTTP/1.1" 200 OK

```

In this way backend works

Step 4: Real-Time Interview Interaction

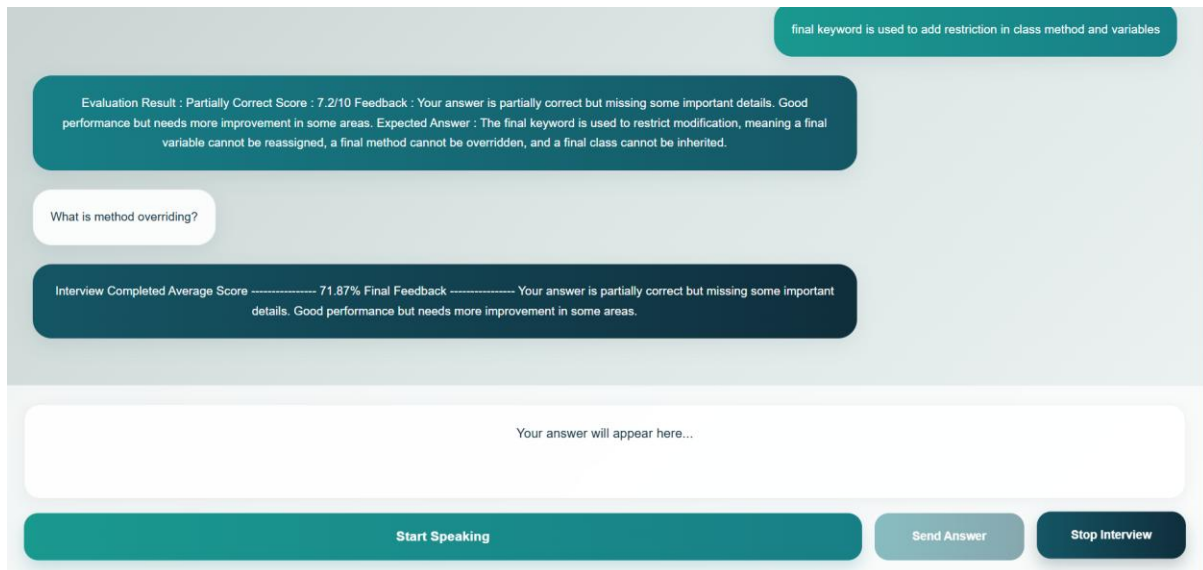
Questions are displayed through a chat-based interview interface. The platform also supports speech synthesis to narrate questions and voice-based answer input and create a more interactive interview experience.



Interview Session

Step 5: Answer Submission

- Users submit answers through voice-based answer interaction. The submitted responses are sent to the backend for evaluation.



Final evaluation result after interview

Step 6: Semantic Evaluation

The NLP evaluation layer processes the response:

- SBERT calculates semantic similarity between the user's answer and expected answers.

Internal working of all-MiniLM-L6-v2 :

Step 1: Input Text Processing

The model first receives two sentences as input.

Sentence 1:

A stack is a linear data structure that follows the Last In First Out (LIFO) principle.

Sentence 2:

A Stack works on the LIFO process that is the element inserted last will be removed first.

Step 2: Tokenization

The tokenizer breaks the sentences into smaller units called tokens.

Example :

Sentence 1 Tokens : ["a", "stack", "is", "a", "linear", "data", "structure"]

Sentence 2 Tokens : ["a", "stack", "works", "on", "the", "lifo", "process"]

The tokenizer also converts tokens into token IDs.

Example :

"stack" = 4567

"lifo" = 8932

These IDs are understandable by the neural network.

Step 3: Word Embedding Generation

Each token is converted into a dense numerical vector called an embedding.

Example :

"stack" = [0.12, -0.45, 0.91, ...]

Words with similar meanings produce similar vector representations.

Example

"LIFO" $\approx$ "Last In First Out"

This allows the model to understand semantic meaning rather than exact keyword matching.

Step 4: Transformer Encoder Processing

The embeddings are passed through the MiniLM transformer encoder.

The 'all-MiniLM-L6-v2' model contains:

1. 6 transformer layers
2. Self-attention mechanism
3. Contextual language understanding
4. The transformer learns relationships between words.

Example :

In the sentence: "last inserted removed first"

The attention mechanism understands that:

"inserted" relates to "removed"

"last" relates to "first"

Overall meaning represents the LIFO principle

Step 5: Contextual Understanding

After passing through transformer layers, embeddings become context-aware.

Example :

The word : "stack"

in a programming sentence gets a different meaning from : "stack of books"

The model understands context automatically.

Step 6: Sentence Embedding Creation

SBERT converts the entire sentence into a single fixed-length vector called a sentence embedding.

Example :

Sentence 1 : [0.23, -0.11, 0.77, ...]

Sentence 2 : [0.25, -0.09, 0.80, ...]

The embedding size for `all-MiniLM-L6-v2` is : 384 dimensions

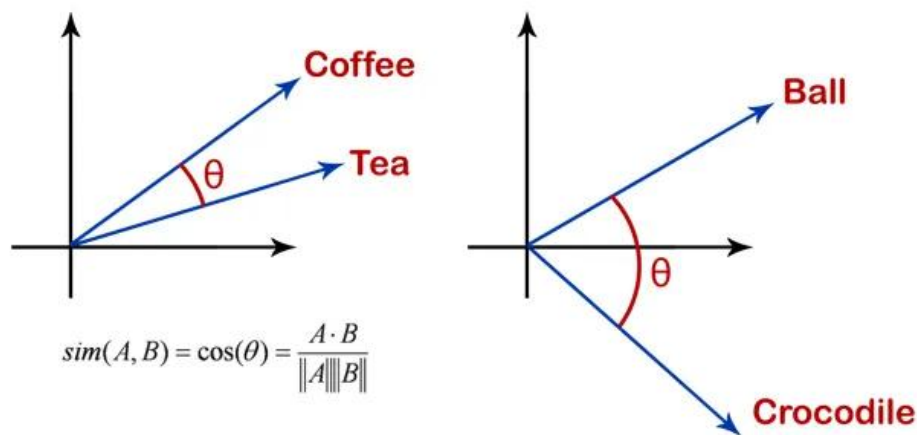
These embeddings capture the overall meaning of the sentence.

Step 7: Cosine Similarity Calculation

The generated sentence embeddings are compared using cosine similarity.

Formula

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$



Where:

A= Expected answer vector

B= User answer vector

Step 8: Semantic Matching

Even though both sentences use different wording, the model understands they express the same meaning.

Expected Answer : "Last In First Out principle"

User Answer : "LIFO process"

The model identifies them as semantically similar.

Similarly: "last inserted removed first"

is understood as: "last element added first removed"

Therefore, the cosine similarity score becomes high.

Example Similarity Score : 0.82 - 0.95

A higher score indicates stronger semantic similarity.

Step 9: Final Decision

The system compares the similarity score with a predefined threshold.

Example :

if similarity > 0.75:

return "Correct Answer"

else:

return "Incorrect Answer"

If the similarity score exceeds the threshold, the answer is considered correct.

- RoBERTa performs logical inference validation to determine conceptual correctness.

Internal working of roberta-large-mnli :

Step 1: Input Pair Creation

Unlike SBERT, RoBERTa-large-MNLI processes two sentences together.

The sentences are arranged as:

Premise:

A stack is a linear data structure that follows the Last In First Out principle.

Hypothesis:

A Stack works on the LIFO process where the element inserted last is removed first.

Here:

Expected Answer : Premise

User Answer : Hypothesis

Step 2: Byte-Pair Encoding (BPE) Tokenization

The RoBERTa tokenizer breaks text into subword tokens using Byte-Pair Encoding (BPE).

Example :

"inserted" = ["insert", "ed"]

"removal" = ["remove", "al"]

This helps the model understand unknown or complex words.

Special tokens are also added : <s> Premise </s></s> Hypothesis </s>

These tokens help the transformer identify sentence boundaries.

Step 3: Token Embedding Generation

Each token is converted into a high-dimensional vector.

Example :

"stack" = [0.34, -0.18, 0.91, ...]

The model uses:

1. Token embeddings
2. Positional embeddings
3. Attention-based contextual embeddings

These embeddings represent semantic meaning numerically.

Step 4: Transformer Encoder Processing

The token embeddings pass through RoBERTa-large transformer layers.

The transformer uses self-attention to understand relationships between words.

Example :

In the sentence : "last inserted removed first"

The model learns :

"last" is related to "first"

"inserted" relates to "removed"

Entire sentence expresses LIFO behavior

Step 5: Contextual Representation Learning

RoBERTa creates contextual representations for each token.

Example :

The word : "stack"

gets different meanings depending on context: "stack data structure" vs "stack of books"

This contextual understanding improves semantic analysis.

Step 6: Sentence Relationship Understanding

The model compares both sentences internally.

It analyzes whether the hypothesis logically follows from the premise.

Example :

Expected Answer :

"Last In First Out principle"

User Answer :

"element inserted last is removed first"

The model understands both express the same concept.

Step 7: Classification Layer

After transformer processing, the final representation is passed into a classification layer.

The model predicts probabilities for three classes :

Entailment : User answer supports expected answer

Neutral : Partial or unclear relation

Contradiction : Contradiction

Step 8: Softmax Probability Calculation

The classifier produces probability scores.

Example Output :

Entailment : 0.94

Neutral : 0.04

Contradiction : 0.02

Since entailment has the highest probability, the answer is considered correct.

Step 9: Final Decision

The application checks the entailment score.

Example :

```
if entailment_score > 0.80 :
```

```
    return "Correct Answer"
```

```
else:
```

```
    return "Incorrect Answer"
```

A higher entailment score indicates stronger semantic agreement.

Step 7: Score Generation and Adaptive Difficulty

Based on evaluation results, similarity scores are generated. The adaptive difficulty engine then updates the interview difficulty level according to user performance.

```

PS D:\Interverse\backend> python run.py
QUESTION FOUND:
{'id': 61, 'question': 'What is Java?', 'answers': ['Java is a high-level, object-oriented programming language developed by Sun Microsystems that is widely used for building platform-independent applications.', 'It is a programming language and computing platform that allows developers to write code once and run it anywhere using the Java Virtual Machine (JVM).', 'Java is a versatile, class-based language known for its portability, security, and robustness, commonly used in web, mobile, and enterprise applications.', 'It is an object-oriented programming language that follows the principle of write once, run anywhere, making it platform-independent.', 'Java is a popular programming language used to develop applications ranging from desktop software to large-scale enterprise systems.', 'It is a strongly typed, compiled language that runs on the JVM and supports features like inheritance, polymorphism, and encapsulation.', 'Java is a general-purpose language designed to have minimal implementation dependencies and is widely used in Android and backend development.', 'It is a widely adopted programming language known for its cross-platform capability, reliability, and extensive library support.'], 'difficulty': 'easy'}
INFO: 127.0.0.1:64585 - "GET /interview/question/5 HTTP/1.1" 200 OK
=====
USER ANSWER:
Java is a high level programming language Java support object oriented programming Java is platform independent
EXPECTED ANSWERS:
['Java is a high-level, object-oriented programming language developed by Sun Microsystems that is widely used for building platform-independent applications.', 'It is a programming language and computing platform that allows developers to write code once and run it anywhere using the Java Virtual Machine (JVM).', 'Java is a versatile, class-based language known for its portability, security, and robustness, commonly used in web, mobile, and enterprise applications.', 'It is an object-oriented programming language that follows the principle of write once, run anywhere, making it platform-independent.', 'Java is a popular programming language used to develop applications ranging from desktop software to large-scale enterprise systems.', 'It is a strongly typed, compiled language that runs on the JVM and supports features like inheritance, polymorphism, and encapsulation.', 'Java is a general-purpose language designed to have minimal implementation dependencies and is widely used in Android and backend development.', 'It is a widely adopted programming language known for its cross-platform capability, reliability, and extensive library support.']
=====
EVALUATION RESULT:
{'result': 'Correct', 'similarity': 0.8260903308266088, 'score': 82.61, 'score_out_of_10': 8.3, 'predicted_label': 1, 'nli': 'ENTAILMENT', 'matched_answer': 'Java is a high-level, object-oriented programming language developed by Sun Microsystems that is widely used for building platform-independent applications.', 'feedback': 'Your answer is almost correct. Very good performance but you need more improvement.', 'ai_response': '\nResult : Correct\nEvaluation Score : 8.3/10\n\nFeedback :\nYour answer is almost correct. Very good performance but you need more improvement.\n\nExpected Answer :\nJava is a high-level, object-oriented programming language developed by Sun Microsystems that is widely used for building platform-independent applications.\n'}

```

```

=====
UPDATING DIFFICULTY
CURRENT LEVEL: easy
RECENT SCORES: [0.5863136291503906, 0.8260903308266088]
=====
AVERAGE SCORE: 0.7062019799884998
LEVEL INCREASED TO: medium
INFO: 127.0.0.1:56391 - "POST /interview/answer HTTP/1.1" 200 OK
=====
GET QUESTION CALLED
SESSION: 5
DOMAIN: programming languages (java)
DIFFICULTY: medium
=====
FETCHING QUESTION
DOMAIN: programming languages (java)
DIFFICULTY: medium
ASKED: 2
DOMAIN MATCH: 60
DIFFICULTY MATCH: 20
AFTER REMOVE ASKED: 20
QUESTION SELECTED
{'id': 91, 'question': 'What is throws keyword?', 'answers': ['The throws keyword is used in a method signature to declare the exceptions that a method might throw during execution.', 'It is a keyword used to indicate that a method can throw one or more exceptions, which must be handled by the caller.', 'The throws keyword is used to pass the responsibility of exception handling to the calling method.', 'It is used to declare checked exceptions in a method so that the compiler is aware of possible errors.', 'The throws keyword specifies the type of exceptions a method can generate, allowing proper handling.', 'It is used in method definitions to inform that an exception may occur and must be handled externally.', 'The throws keyword helps in propagating exceptions up the call stack instead of handling them immediately.', 'It is a declaration that a method mi

```

In this way backend works

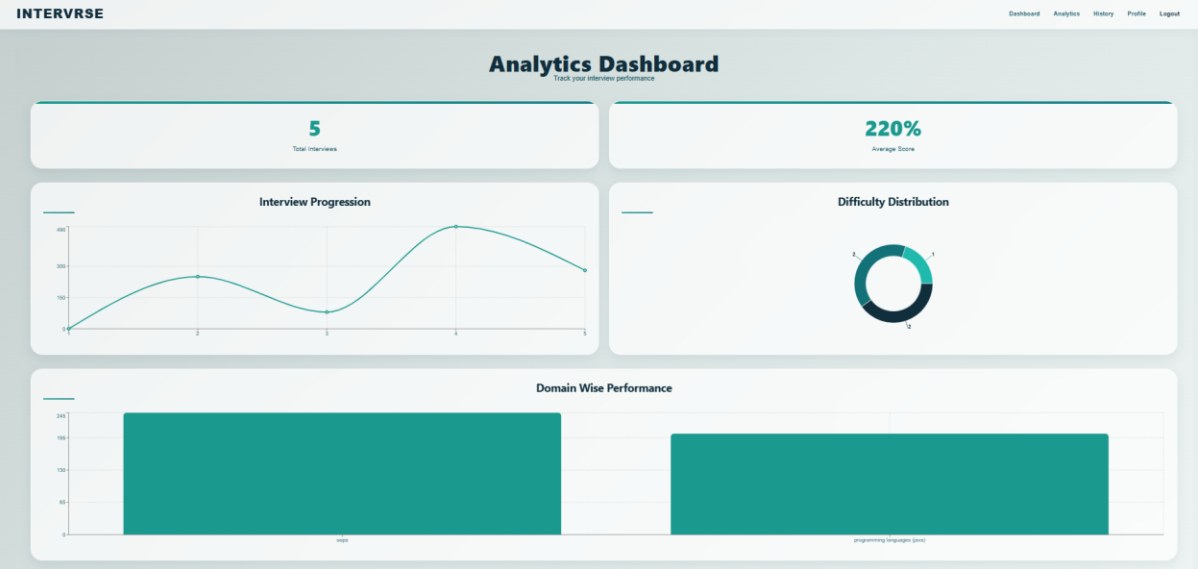
Step 8: Analytics and Result Storage

Interview scores, responses, session details, and analytics are stored in the database. The final results and performance statistics are displayed on the user dashboard for further analysis and improvement.

This architecture enables INTERVERSE to provide a realistic, secure, and intelligent interview preparation environment using modern web technologies and NLP-based evaluation techniques.



History Page



Analytics Page

My Profile

Complete your profile to improve your interview experience.



Profile Completion: 0%

Complete remaining details to improve your profile.

Upload Photo

Name	Email
Phone	Education
College	Skills
LinkedIn	GitHub
Address	
Save Profile	

Profile Page

k.Results and Discussion

The developed INTERVERSE platform successfully performs semantic answer evaluation, adaptive interview management, secure authentication, and real-time interview interaction. The system combines transformer-based Natural Language Processing models with a full-stack web architecture to simulate realistic technical interview environments.

The platform was tested using multiple domain-specific interview sessions across different difficulty levels. The integration of semantic similarity analysis and logical inference significantly improved contextual answer evaluation compared to traditional keyword-based systems. The system was also able to dynamically update interview difficulty according to user performance, creating a more personalized interview experience.

Major Achievements

1. Interactive Interview Flow

The system successfully conducts interviews through a conversational chat-based interface. Questions are dynamically fetched from the dataset according to the selected domain and current difficulty level. The real-time interaction between frontend and backend creates a smooth interview experience similar to an actual technical interview environment.

2. Semantic Answer Evaluation

One of the major achievements of the project is contextual answer evaluation using SBERT and RoBERTa models. Instead of checking only exact keywords, the system evaluates conceptual similarity and logical correctness of responses. This allows users to provide flexible and descriptive answers while still receiving accurate evaluation scores.

3. Adaptive Difficulty Progression

The adaptive engine dynamically modifies interview difficulty based on previous performance scores. Users who consistently perform well receive more advanced questions, while users with lower scores receive easier questions. This adaptive mechanism improves personalization and interview realism.

4. Authentication and Security

JWT-based authentication successfully secures interview sessions and protected routes. Passwords are securely hashed using PBKDF2-SHA256 before being stored in the database. This improves system security and prevents unauthorized access to user data and interview history.

5. Analytics Dashboard

The analytics dashboard provides users with detailed interview statistics and performance analysis. Users can monitor:

- Average interview scores

- Interview history
- Difficulty progression
- Performance trends
- Session-wise analytics

The graphical visualization of results improves understanding of strengths and weaknesses across interview sessions.

6. Speech Interaction Features

The platform supports speech synthesis using browser-based Speech APIs. Questions can be narrated automatically and users can also submit answers through speech input, improving interview realism and user interaction. This feature also enhances accessibility and user engagement during interview sessions.

Discussion

The project demonstrates the practical implementation of transformer-based NLP models in automated interview evaluation systems. The integration of semantic similarity analysis and natural language inference improves contextual understanding and reduces the limitations of traditional keyword-matching systems.

The combination of React.js and FastAPI provided efficient communication between frontend and backend modules. The modular backend architecture improved scalability and maintainability of the system. Dynamic question selection and adaptive difficulty management also contributed to creating a more realistic and structured interview environment.

The use of SBERT improved semantic evaluation by understanding conceptual similarity between answers, while RoBERTa improved logical validation through Natural Language Inference. The hybrid evaluation strategy increased overall scoring accuracy and reliability.

7. Performance Evaluation Metrics

To evaluate the effectiveness of the semantic answer assessment system, INTERVERSE was tested using a manually prepared dataset containing both valid and invalid interview responses from multiple technical domains. The outputs generated by the system were compared with predefined expected labels using standard machine learning evaluation metrics.

The evaluation process measured:

- 1) Accuracy
- 2) Precision
- 3) Recall
- 4) F1-Score
- 5) Confusion Matrix

The obtained performance results are presented below:

```
● PS D:\Interverse\backend> python evaluation_metrics.py
```

```
MODEL EVALUATION RESULTS :
```

```
Accuracy : 0.96
```

```
Precision : 1.00
```

```
Recall : 0.94
```

```
F1 Score : 0.97
```

```
Confusion Matrix:
```

```
[[43  0]  
 [ 4 62]]
```

Result Interpretation

The experimental results demonstrate that the semantic evaluation mechanism performs with a high level of accuracy and consistency. The achieved accuracy score of 96% indicates that the system can correctly classify most user responses during interview sessions.

The precision value of 1.00 shows that responses predicted as correct were highly reliable and produced no false positive classifications. The recall score of 0.94 indicates that the system was able to identify the majority of correct answers while missing only a small number of valid responses.

The F1-score of 0.97 confirms a strong balance between precision and recall, demonstrating the effectiveness of combining SBERT-based semantic similarity analysis with RoBERTa-based logical inference validation.

These results verify that the proposed transformer-based evaluation approach provides more accurate contextual understanding and response assessment compared to traditional keyword-matching techniques commonly used in automated interview systems.

Challenges Faced During Development

Several technical challenges were encountered during the development process:

- Dataset preprocessing and formatting inconsistencies
- NLP model loading time and memory usage
- Frontend-backend synchronization during interview flow
- Managing adaptive difficulty progression accurately
- Preventing repeated questions during sessions
- Handling real-time speech interaction features

- Optimizing API response performance

Despite these challenges, the final system successfully achieved stable interview flow, contextual evaluation, and analytical feedback generation.

I. Future Scope

The current version of INTERVERSE successfully provides an AI-powered interview environment with semantic answer evaluation, adaptive questioning, voice interaction, and performance analytics. Although the platform already supports realistic technical interview practice, several advanced enhancements can be implemented in the future to further improve system intelligence, scalability, and user experience.

One possible enhancement is the integration of more advanced conversational AI models capable of generating context-aware follow-up questions during interviews. At present, the system selects questions dynamically from predefined datasets, but future versions may support fully interactive interview conversations where the next question depends on the candidate's previous response and performance.

The voice interaction module can also be expanded further. Currently, the platform supports browser-based speech recognition and speech synthesis. Future implementations may include more accurate speech-to-text systems, real-time voice communication, and pronunciation analysis to improve communication skill assessment.

Another important improvement area is behavioral and confidence analysis using computer vision techniques. Webcam-based monitoring systems can be integrated to analyze facial expressions, eye contact, confidence levels, and emotional behavior during interviews. This would help simulate real interview environments more effectively and provide additional feedback related to soft skills and communication.

The platform can also be upgraded to support cloud-based deployment and scalable infrastructure for handling large numbers of concurrent users. Migrating the system to cloud services would improve reliability, scalability, and accessibility for educational institutions and recruitment organizations.

Future versions of INTERVERSE may additionally include multilingual interview support to allow users from different linguistic backgrounds to practice interviews in their preferred language. Mobile application integration can also improve accessibility and allow users to attend interview sessions from smartphones and tablets.

Several advanced analytical features may also be incorporated, such as detailed skill-gap analysis, personalized interview recommendations, performance prediction, and domain-specific learning suggestions. Integration with recruitment systems and online hiring platforms may further extend the practical usability of the project.

Possible future enhancements include:

- 1) Dynamic AI-generated follow-up questioning
- 2) Advanced speech-to-text and voice analysis
- 3) Webcam-based emotion and confidence detection
- 4) Cloud-based scalable deployment
- 5) Multi-language interview support

- 6) Personalized performance recommendations
- 7) Mobile application development
- 8) Integration with recruitment platforms
- 9) Enhanced analytics and reporting systems
- 10) Larger and continuously updated interview datasets

With these future improvements, INTERVERSE can evolve into a more intelligent, scalable, and comprehensive interview preparation and assessment platform suitable for both academic and professional applications.

m. Conclusion

INTERVERSE successfully demonstrates the development of an intelligent web-based interview platform that combines full-stack web technologies with transformer-based Natural Language Processing techniques to create an interactive interview preparation environment. The platform is capable of conducting technical interview sessions, evaluating answers contextually, adapting question difficulty according to user performance, and providing analytical feedback to users.

The project integrates React.js for frontend development and FastAPI for backend services, enabling a responsive and modular system architecture. The application supports secure authentication, interview session management, real-time interaction, voice-assisted communication, and interview performance tracking. The use of SQLite and SQLAlchemy further helps in managing user records, interview history, and analytical data efficiently.

A major strength of the platform is the implementation of semantic answer evaluation using SBERT and RoBERTa models. Instead of depending on simple keyword matching, the system evaluates the contextual meaning of user responses through semantic similarity analysis and natural language inference. This approach improves evaluation quality and allows the system to recognize conceptually correct answers even when users provide responses using different wording styles.

The adaptive difficulty mechanism also enhances the realism of the interview process by dynamically adjusting question complexity based on candidate performance. In addition, features such as speech-based question narration, interview analytics, and history tracking improve overall user engagement and learning experience.

The platform currently provides:

- Semantic and contextual answer evaluation
- Adaptive interview difficulty progression
- Secure authentication and protected access
- Real-time interview interaction
- Voice-assisted interview functionality
- Interview analytics and performance tracking
- Domain-based technical interview practice

The project highlights the practical application of transformer-based NLP models in educational and recruitment-oriented systems. It also demonstrates how modern AI techniques can be integrated with full-stack web development to build intelligent and scalable interview preparation platforms.

Although the current implementation achieves its primary objectives successfully, there is significant scope for future enhancement through conversational AI, cloud deployment, advanced speech interaction, webcam-based behavioral analysis, and AI-generated follow-up questioning.

Overall, INTERVERSE represents a successful implementation of an AI-powered interview simulation system and shows how semantic understanding, adaptive learning, and modern web technologies can work together to create a realistic and effective interview preparation platform.

n. References

- [1] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019.
- [2] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” arXiv preprint arXiv:1907.11692, 2019.
- [3] L. Järvillehto, Y. Sun, N. Aiba, S. Haginoya, H. Hallström, J. Korkman, and P. Santtila, “Large Language Model (LLM) and Human Performance in Child Investigative Interviewing Question Formulation Tasks,” Behavioral Sciences & the Law, vol. 44, pp. 142–163, 2026.
- [4] A. Wuttke, M. Aßenmacher, C. Klamm, M. M. Lang, Q. Würschinger, and F. Kreuter, “AI Conversational Interviewing: Transforming Surveys with LLMs as Adaptive Interviewers,” 2025.
- [5] H. Zhang, Y. Liu, X. Guan, J. Cai, and J. M. Carroll, “Harnessing the Power of AI in Qualitative Research: Role Assignment, Engagement, and User Perceptions of AI-Generated Follow-Up Questions in Semi-Structured Interviews,” 2025.
- [6] FastAPI Documentation, “FastAPI Framework Documentation.” [Online]. Available: <https://fastapi.tiangolo.com/>
- [7] React Documentation, “React Official Documentation.” [Online]. Available: <https://react.dev/>
- [8] Hugging Face Documentation, “Transformers Library Documentation.” [Online]. Available: <https://huggingface.co/docs/transformers/>
- [9] SQLAlchemy Documentation, “SQLAlchemy ORM Documentation.” [Online]. Available: <https://www.sqlalchemy.org/>
- [10] Pandas Documentation, “Pandas Data Analysis Library.” [Online]. Available: <https://pandas.pydata.org/>
- [11] JWT Introduction, “JSON Web Token Documentation.” [Online]. Available: <https://jwt.io/>